

MiniLang Pipeline. Decisiones de lenguaje y paradigmas

Examen parcial, Parte B.

Angélica Salazar Delgado.

Etapas y paradigmas

Cada etapa usa el lenguaje cuyo paradigma mejor se ajusta a su tarea, y las tres se comunican solo mediante archivos.

Etapas	Lenguaje	Paradigma	Responsabilidad	Salida
1	Java	Orientado a objetos	Análisis léxico, sintáctico y semántico, y traducción	programa.ir
2	Python	Funcional	Ejecutar FILTER, MAP y REDUCE	resultado.txt
3	MIPS	Imperativo de bajo nivel	Calcular la firma de verificación	firma.txt

1. ¿Por qué Java resulta adecuado para la etapa de análisis y modelado de instrucciones?

Java es adecuado porque su sistema de clases permite representar la estructura del lenguaje en el código. La clase abstracta Operacion agrupa a FilterInstr, MapInstr y ReduceInstr, igual que <operacion> agrupa a <filter>, <map> y <reduce> en la gramática.

El polimorfismo evita preguntar por el tipo de cada instrucción. Main recorre la lista y llama a toIR(), y cada subclase genera su propia línea. El análisis semántico consulta requiereLista() y produceEscalar() en lugar de usar instanceof, así que una operación nueva solo requiere una subclase.

Además, el tipado estático impide compilar una subclase que no implemente toIR(), y la biblioteca estándar ofrece Pattern y Matcher para definir cada token con una expresión regular.

2. ¿Qué cambia conceptualmente entre describir una transformación con estilo imperativo y funcional?

El estilo imperativo describe cómo se obtiene el resultado, mediante instrucciones que modifican un estado. El funcional describe qué transformación se aplica, con funciones que reciben datos y devuelven datos nuevos.

El proyecto usa los dos. El lexer de Java avanza con pos++ y cuenta líneas con linea++, y MIPS lleva esto al extremo, con registros que se sobrescriben y ciclos hechos con saltos condicionales. En Python, filter(lambda x: comparar(x, valor), datos) dice qué elementos se conservan, sin índices ni variables que cambien. Incluso la secuencia de operaciones es un reduce. Como ninguna función altera la lista que recibe, cada una se prueba por separado y no hay efectos ocultos. A cambio, el estilo imperativo da control directo sobre la memoria y el orden de ejecución, que es lo que necesita MIPS.

3. ¿Qué información se pierde o se conserva al convertir programa.mini a programa.ir?

Se conserva todo lo necesario para ejecutar el programa, el orden de las instrucciones, el tipo de cada operación con su operador y su operando, y los valores de DATA.

Se pierde lo que solo servía para escribirlo o para reportar errores.

Los números de línea. Por eso el error del caso 8, que ocurre en Python, no puede señalar una línea de programa.mini.

Los espacios, los saltos de línea y la forma escrita de los números, así 007 llega a Python como 7. La estructura de objetos, que existe solo dentro de Java. La garantía de validez. Por eso Python vuelve a validar el contrato. A cambio, el formato gana precisión. Cada línea tiene una

instrucción con campos separados por |, y Python la interpreta con un split sin necesitar otro lexer.

4. ¿Por qué la representación intermedia puede compararse con una fase de un compilador?

Porque programa.ir cumple el papel de la representación intermedia de un compilador, que separa el análisis del lenguaje de su ejecución. Java es la parte frontal, con las fases léxica, sintáctica y semántica, y Python es un intérprete que ejecuta esa representación, así el pipeline combina traducir e interpretar.

La representación intermedia desacopla las etapas. Java no sabe quién ejecutará programa.ir, y Python no sabe cómo se escribió el programa original, así que Python podría reemplazarse por otro ejecutor sin tocar Java. Además, el IR es más fácil de optimizar que el código fuente. Por ejemplo, MAP|+|1 seguido de MAP|+|2 podría combinarse en MAP|+|3.

5. ¿Qué ventajas y costos aparecen al integrar tres lenguajes en lugar de resolver todo con uno?

La principal ventaja es que cada etapa usa el paradigma que mejor la resuelve, y el principal costo es mantener un contrato en cada frontera entre lenguajes.

Ventajas

Las etapas quedan desacopladas y se probaron por separado.

Los errores quedan aislados, y el pipeline indica en qué etapa se detuvo.

Costos

Tres entornos que instalar, JDK, Python y MARS, y un script que los coordine.

Conversiones de texto en cada frontera. En MIPS, leer el número 60 exige recorrer caracteres y restar 48 a cada uno, algo que en Python es un int().

Python usa enteros sin límite y MIPS usa 32 bits, así que un resultado muy grande da la vuelta al llegar a MIPS.

Los tres lenguajes son Turing-completos, pero eso solo dice qué se puede calcular, no qué tan clara o mantenible queda la solución. En un sistema real de este tamaño, un solo lenguaje sería más simple de mantener. Aquí la separación se justifica porque cada etapa muestra un paradigma donde mejor funciona.